

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

College of Computing and Data Science

SC2006 – Software Engineering

Lab 3 Deliverables – AI Report

SCSJ Group 3

Name	Matriculation No.
Xu Hanwen	U2522613H
Teo Kok Wei, Oscar	U2410894D
K Eswaran	U2521677A
Mao Yikai	U2520028F
Tan Leong Jun Joseph	U2321339H

(1) Description of uploaded material

Uploaded Crowdy's Lab 1 system description (a Singapore-only real-time crowd-checking app aggregating data from LTA DataMall and Google Popular Times, with GPS-based map discovery, user accounts and favourites) and the Lab 3 design class diagram. Unlike the Lab 2 upload, which only showed entity classes, the Lab 3 diagram covers the full design: start-up classes (CrowdyApp, CrowdyServer); boundary classes for each screen; control classes for the app and server (e.g. ApiClient, PopularTimesService, PopularTimesScheduler, DirectionsController, LanguageController); integration classes for external services (LtaClient, GoogleMapsScraper, OneMapClient); container classes (PopularTimesCache, SavedVenueIds); and the entity classes, with attributes, operations and design patterns (Facade, Adapter, Singleton, Observer, Repository). The same AI model as Lab 2 (Claude) was used.

(2) Tech-stack prompt and AI response

Prompt:

Based on the uploaded design materials, recommend a technology stack appropriate for an entry-level software engineering team.

Response:

Your class diagram already implies an Expo app talking to an Express server through one ApiClient, with adapters for external services. A stack that fits, for an entry-level team:

- Mobile app: React Native + Expo (TypeScript). Use react-native-maps for the map, Expo Location for GPS, Expo SecureStore for the session token, and i18next for the language switch.
- State: React Context for the session and language; add TanStack Query or Zustand only if needed.
- Backend: Node.js + Express (TypeScript), sharing types such as Venue with the app.
- Scheduling: node-cron for the scraping scheduler; BullMQ + Redis if you need retries and queues.
- Scraping: Puppeteer or Playwright behind the GoogleMapsScraper adapter. Expect it to be fragile.
- Database: MySQL or PostgreSQL via Prisma or Knex, on a managed host (Aiven, Railway, PlanetScale).
- Caching: Redis if you run several server instances; otherwise an in-memory cache.
- External APIs: LTA DataMall (crowds) and OneMap (routing), through their adapters.
- Auth: JWT + bcrypt, enforced by the RequireAuth middleware.
- Testing/CI: Jest or Vitest, Supertest for API routes, ESLint/Prettier, GitHub Actions.
- Hosting: Render or Railway for the backend; Expo EAS for app builds later.

(3) Comparison with Lab 2 (3.3.2)

The AI's recommendations were mostly the same compared with Lab 2. Both recommended React Native with Expo, Node.js with Express, MySQL, JWT with bcrypt. The main differences were some new libraries suggested, dropped or added one/two technologies and some different hosting choice.

Our prompt and technology preferences did not change, so the differences most likely come from the updated class diagram. The Lab 2 diagram contained only entity classes, so the AI could only give general advice. The Lab 3 diagram names specific control and integration classes, such as PopularTimesScheduler, a job scheduler library; LanguageController, a translation library; GoogleMapsScraper, a headless browser tool, which led the AI to recommend some tools for those responsibilities.

(4) Skeleton Prompt and AI response

Prompt:
Using the uploaded class diagram, generate a minimal skeleton template for our project in TypeScript: an Expo (React Native) mobile app and a Node.js/Express server. Produce a folder hierarchy of files that are empty except for TODO comments describing what each file should contain. Do not put any code inside the files: no imports, classes, functions or types.

Return the result as a downloadable .zip file.

Response:

```
crowdy-skeleton/
├── app/
│   ├── App.tsx                 // TODO: start-up, providers, navigation
│   └── src/
│       ├── boundary/           MainUI, MapUI, CrowdSnapshotUI, SearchUI,
│       │                       SavedUI, VenueDetailUI, DirectionsUI, SettingsUI
│       ├── control/            ApiClient, SessionController, SavedVenuesController,
│       │                       SearchController, LocationController, LanguageController
│       └── entity/             Venue, Route, RouteStep, MapRegion
└── server/src/
    ├── CrowdyServer.ts         // TODO: mount routes, start scheduler
    ├── controllers/            Venue, Directions, Heatmap, Auth, Favourites
    ├── services/               CrowdAggregator, PopularTimesService, PopularTimesScheduler, ...
    ├── integration/            LtaClient, GoogleMapsScraper, OneMapClient, BrowserManager, ...
    ├── middleware/             RequireAuth
    └── entity/                 Venue, Station, Account, Favourite, PopularTimesPattern, ...
```

Abbreviated. The .zip also includes container folders and a README.

(5) Critique (3.3.4)

To what extent do we agree with its choices?

We partly agree. It separated the app and server, and created one file per class from the diagram. However, it grouped files by boundary/control/entity folders, whereas our code groups them by type (screens, services, routes), which suits React Native and Express conventions better.

Has it missed something obvious or failed to produce anything reasonable?

No test folders, no configuration for API keys (.env), no shared types between app and server, no navigation setup, no fixed venue data files.

Did it add anything beyond what was requested?

No. Every file contains only TODO comments. The only extra was a README explaining that the folder is an AI-generated skeleton, not the real application code.